

DOM Manipulation

Control Web Pages Dynamically with JavaScript

■ DOM

■ LEARNING OUTCOME

By the end of this module you will select, create, modify, and remove HTML elements, understand the document tree structure, and build dynamic UI components entirely with JavaScript.

■ Skills You Will Gain

- Select with `querySelector` and `querySelectorAll`
- Read and write: `textContent`, `innerHTML`
- Add, remove, toggle CSS classes
- Create, insert, and remove elements
- Set attributes and dataset properties
- Traverse the DOM: parent, children, siblings

■ DOM Manipulation IS front-end development. Every interactive feature -- modals, dropdowns, tabs, live search -- is DOM manipulation. Master this and you can build anything.

SECTION 1

Selecting Elements

```
// querySelector -- first match (or null) const hero = document.querySelector('.hero'); // class
const title = document.querySelector('#main-title'); // ID const btn =
document.querySelector('button[type="submit"]'); const nested = document.querySelector('.card
.btn-primary'); // querySelectorAll -- ALL matches (NodeList) const allCards =
document.querySelectorAll('.card'); const allLinks = document.querySelectorAll('nav a'); //
NodeList is NOT an array -- convert first! const cardsArr = [...allCards]; cardsArr.forEach(card =>
card.classList.add('visible')); // Search WITHIN a found element (scoped search) const form =
document.querySelector('#contact-form'); const inputs = form.querySelectorAll('input'); // only
inputs in this form // TRAVERSAL from element const el = document.querySelector('.card');
el.parentElement // parent element el.children // HTMLCollection of direct children
el.firstChild // first child element el.lastElementChild // last child element
el.nextElementSibling // next sibling el.previousElementSibling // previous sibling
el.closest('.container') // nearest ancestor matching selector
```

SECTION 2

Reading, Writing, Classes & Attributes

```
const el = document.querySelector('.card'); // TEXT CONTENT -- safe, no HTML parsing el.textContent
= 'New text <b>not bold</b>'; // shown literally // innerHTML -- parses HTML (NEVER use with
user-provided content -- XSS!) el.innerHTML = '<strong>Bold</strong> and <em>italic</em>'; // Input
value document.querySelector('#email').value = 'test@example.com'; // CLASSES
el.classList.add('active'); // add el.classList.remove('hidden'); // remove
el.classList.toggle('open'); // add if missing, remove if present el.classList.contains('active');
// true/false el.classList.replace('old','new'); // swap el.classList.add('a','b','c'); // multiple
at once // ATTRIBUTES el.getAttribute('href') // get el.setAttribute('href','/new') // set
el.removeAttribute('disabled') // remove el.hasAttribute('required') // boolean check // data-*
attributes el.dataset.userId = '42'; // sets data-user-id='42' el.dataset.theme = 'dark'; // sets
data-theme='dark' const id = el.dataset.userId; // read it back // STYLES el.style.color =
'#E44D26'; el.style.display = 'none'; // hide el.style.display = 'block'; // show // Get COMPUTED
styles (includes CSS stylesheet) const computed = window.getComputedStyle(el);
computed.getPropertyValue('font-size'); // '16px'
```

SECTION 3

Creating and Inserting Elements

```
// CREATE element safely (no XSS risk) function createCard({ title, description, url }) { const
article = document.createElement('article'); article.className = 'card'; const h3 =
document.createElement('h3'); h3.textContent = title; // textContent is safe! const p =
document.createElement('p'); p.textContent = description; const a = document.createElement('a');
a.href = url; a.textContent = 'Read more'; a.target = '_blank'; a.rel = 'noopener';
article.append(h3, p, a); // append multiple at once return article; } // INSERTING const grid =
document.querySelector('.grid'); grid.appendChild(card); // end grid.prepend(card); // start
grid.append(card1, card2, 'text'); // multiple + text nodes // insertAdjacentElement
el.insertAdjacentElement('beforebegin',newEl); // before el
el.insertAdjacentElement('afterend',newEl); // after el
el.insertAdjacentElement('afterbegin',newEl); // first child
el.insertAdjacentElement('beforeend',newEl); // last child // REMOVING card.remove(); // remove
from DOM grid.removeChild(card); // remove specific child // DocumentFragment -- batch insert (one
reflow not N) const frag = document.createDocumentFragment(); products.forEach(p =>
frag.appendChild(createCard(p))); grid.appendChild(frag); // one DOM update!
```

■ PRACTICE Q & A

Q1. What is the difference between `textContent` and `innerHTML`?

A: `textContent`: sets/gets plain text -- HTML tags shown as literal text. `innerHTML`: parses HTML and renders it. ALWAYS use `textContent` when inserting user content to prevent XSS attacks.

Q2. How do you efficiently add 100 items to the DOM without 100 repaints?

A: Use `DocumentFragment`: create all elements inside fragment (detached from DOM), then append fragment to real DOM in one operation. One reflow instead of 100.

Q3. What does `el.closest()` do?

A: Traverses UP from the element returning the nearest ancestor (or self) matching the selector. `el.closest('.card')` finds the containing card. Perfect for event delegation to identify which card was clicked.

Q4. What is the difference between `el.children` and `el.childNodes`?

A: `children`: `HTMLCollection` of element children only (no text nodes). `childNodes`: `NodeList` of ALL nodes including text nodes and comments. Use `children` for element traversal.

Q5. Why should you avoid `innerHTML` with user-provided content?

A: XSS (Cross-Site Scripting): if user provides `<script>stealCookies()</script>` as content, `innerHTML` executes it. Use `textContent` for user data -- it's always safe.

■ DOM Cheat Sheet	
<code>document.querySelector('.cls')</code>	First matching element
<code>[...document.querySelectorAll('li')]</code>	All matching -- convert to array
<code>el.textContent = 'safe text'</code>	Set text -- no HTML parsing
<code>el.classList.add/remove('cls')</code>	Add or remove CSS class
<code>el.classList.toggle('open')</code>	Toggle class on/off
<code>el.classList.contains('active')</code>	Check if class exists
<code>el.setAttribute('src', 'img.jpg')</code>	Set HTML attribute
<code>el.dataset.userId</code>	Read/write data-* attributes
<code>document.createElement('div')</code>	Create new element
<code>parent.append(child1, child2)</code>	Add multiple children
<code>el.closest('.card')</code>	Find nearest matching ancestor
<code>createDocumentFragment()</code>	Batch DOM updates